```
"""Run the simulation.  Quick and dirty style, compared to other modules..."""

import os, sys

if 'city' in sys.modules:
    reload(sys.modules['city'])
from city import *
from basic import *
from multi import *
from utils import *


outdir = 'results4'
def outname(fname):
    """Return file name in output directory."""
    from os.path import join
    return join(outdir, fname)


class Log(object):

    """File-like object writing to both stdout and a log file."""

    def __init__(self, fname='log'):
        """Open log file `fname`.log; read old headers into self.done."""
        fname = outname(fname + '.log')
        self.done = []
        try:
            f = file(fname)
            for line in f:
                if line.startswith('==> '):
                    self.done.append(line[4:line.index(' <==')])
            f.close()
        except IOError:
            pass
        self._file = file(fname, 'a')

    def write(self, s):
        sys.stdout.write(s)
        self._file.write(str(s))
        # Make progress be immediately visible in log.
        self._file.flush()

    def close(self):
        self._file.close()


def header(s):
    """Return decorated string."""
    return '\n==> ' + s + ' <=='


def plot(sets, fname='plot'):
    """Plot into `fname`.ps the given lists (as function of index).

    `sets` is a list of pairs (setname, list) to plot.  setname.dat and fname.plt
    files are produced.

    """
    datnames = []
    for name, values in sets:
        datname = name + '.dat'
        datnames.append(datname)
        f = open(outname(datname), 'w')
        for v in values:
            f.write("%s\n" % v)
        f.close()
    f = open(outname(fname + '.plt'), 'w')
    f.write("set terminal postscript color\n"
            "set data style lines\n" +
```

```
            "set output '%s.ps'\n" % fname +
            "set title '%s'\n" % fname +
            "plot '%s'\n" % "', '".join(datnames))
    f.close()
    os.system('(cd %s; gnuplot < %s.plt)' % # gv %s.ps was really annoying
              (outdir, fname))


# Common params for tests
turns = {0: 60, 1: 15, 2: 05, -1: 20}


def implement(plan, load=0.2, turns=turns, use_uniload=1, filters=[rand_filter], statqueue=0):
    """Simple specification.

    - `statqueue` can be 0 (plain), positive integer (giving LoadHistQueue
      history length) or negative integer (giving LoadExpQueue characteristic
      decay length).

    """
    c = City()
    if statqueue > 0:
        c.make_queue = lambda : c.LoadHistQueue(statqueue)
    elif statqueue < 0:
        c.make_queue = lambda : c.LoadExpQueue(-statqueue)
    if use_uniload:
        def router(ports):
            return c.RandomRouter(uniload(turn_probs(turns, ports)))
    else:
        def router(ports):
            return c.RandomRouter(turn_probs(turns, ports))
    plan.implement(c,
                   source=lambda : c.ProbSource(load),
                   drain=c.StatDrain,
                   server=lambda ports: c.SwitchedServer(filters),
                   router=router)
    return c


# First phase - check simulation consitency and stability

react_loads = [0.05, 0.1, 0.2]

plans = {'5x5': GridPlan(5, 5),
         '10x10': GridPlan(10, 10),
         '15x15': GridPlan(15, 15)
         }
for n, p in plans.items():
    try:
        fname = outname(n + '.plan')
        f = open(fname)
        p.from_str(f.read())
    except IOError:
        f = open(fname, 'w')
        p.full_plan()
        p.sieve_roads(0.3)
        f.write(str(p))
    f.close()
react_plans = plans

policies = {'rand': [rand_filter],
            'lru': [lru_filter],
            'longest': [longest_queue_filter, rand_filter],
            'hist': [last_emptying_filter, rand_filter],
            'inorder': [inorder_filter, rand_filter]
            }
for name, filters in policies.items():
    policies['hold_' + name] = [(hold_filter, [4])] + filters
react_policies = policies
```

```
def run_react(plan, **params):
    """Return 3 history lists and last statistics for sources turned on/off.

    First list tracks number of cars, the second tracks the average waiting
    time (over all queues in the city) and the third - the deviation in the
    waiting time.

    """
    c = implement(plan, **params)
    phases = c.simtime.phases
    car_hist = []
    wait_hist = []
    dev_hist = []
    for i in range(6000):
        if i == 4000:
            phases[Source].clear()              # dirty hack
        c.advance(1)
        car_hist.append(len(phases[Car]))
        stat = Statistics(*[q.waited for q in phases[Queue]])
        wait_hist.append(stat.mean())
        dev_hist.append(stat.deviation())
    return car_hist, wait_hist, dev_hist, stat


def all_react():
    """Run reaction tests."""
    log = Log('react')
    print >>log, header('Plans:')
    for planname, plan in react_plans.items():
        print >>log, header(planname)
        print >>log, plan
    for load in react_loads:
        for filtersname, filters in react_policies.items():
            name = 'react_%.2f-%s' % (load, filtersname)
            car_hists = []
            wait_hists = []
            dev_hists = []
            for planname, plan in react_plans.items():
                datname = name + '_' + planname
                if datname in log.done:
                    continue
                print >>log, header(datname)
                # exponential queue (if needed)
                statqueue = (last_emptying_filter in filters) and -4
                car, wait, dev, last = run_react(plan, load=load,
                                                 filters=filters,
                                                 statqueue=statqueue)
                print >>log, last
                car_hists.append((datname + '-car', car))
                wait_hists.append((datname + '-wait', wait))
                dev_hists.append((datname + '-dev', dev))
            plot(car_hists, name + '-car')
            plot(wait_hists, name + '-wait')
            plot(dev_hists, name + '-dev')
    log.close()


# For the rest, run using conclusions from the above tests: 15x15, 5000
# cycles, 3 plans are good (even on the safe side).
# Also shift interest to higher loads.

main_loads = [0.15, 0.2, 0.23]

plans = dict(zip('ABC', [GridPlan(15, 15) for i in range(3)]))
for n, p in plans.items():
    try:
        fname = outname(n + '.plan')
        f = open(fname)
        p.from_str(f.read())
```

```
    except IOError:
        f = open(fname, 'w')
        p.full_plan()
        p.sieve_roads(0.3)
        f.write(str(p)+'\n')
    f.close()
main_plans = plans


# Used for following tests.
def run_main(plan, **params):
    """Return tuple of (waited_total, waited3, waited4) statistics.

    The last two are separate stats over junctions with 3 and 4 roads.

    """
    c = implement(plan, **params)
    c.advance(5000)
    queues3 = []
    queues4 = []
    for (x, y), dirs in c.plan.grid.items():
        n = len(dirs)
        if n == 3:
            for d in dirs:
                queues3.append(c.grid[x,y].inputs[d])
        elif n == 4:
            for d in dirs:
                queues4.append(c.grid[x,y].inputs[d])
    w3 = Statistics(*[q.waited for q in queues3])
    w4 = Statistics(*[q.waited for q in queues4])
    w = Statistics(w3, w4)
    return w, w3, w4


policies = {}
for holdname, hold in [('free', []),
                       ('hold2', [(hold_filter, [2])]),
                       ('hold4', [(hold_filter, [4])]),
                       ('hold8', [(hold_filter, [8])])]:
    for flexname, flex in ('flex', [attempt_hold_filter]), ('fix', []):
        for wisdomname, wisdom in [('blind', []), # specially treated below
                                   ('nonempty', []),
                                   ('longest', [longest_queue_filter]),
                                   ('hist', [last_emptying_filter]),
                                   ('inorder', [inorder_filter])]:
            for tiename, tie in ('rand', [rand_filter]), ('lru', [lru_filter]):
                if wisdomname == 'blind':
                    nonempty = []
                    if flexname == 'flex':
                        continue       # flex stucks without any pre-filtering
                else:
                    nonempty = [nonempty_filter]
                policies['_'.join([holdname, flexname, wisdomname, tiename])] = (
                    hold + nonempty + flex + wisdom + tie)
main_policies = policies


def all_main():
    """Run reaction tests."""
    log = Log('main')
    print >>log, header('Plans:')
    for planname, plan in main_plans.items():
        print >>log, header(planname)
        print >>log, plan
    # outermost loop over plans, so that first results for one plan are
    # availiable quicker...
    for planname, plan in main_plans.items():
        for load in main_loads:
            for filtersname, filters in main_policies.items():
                name = '%.2f-%s' % (load, filtersname)
```

```
                datname = name + '-' + planname
                if datname in log.done:
                    continue
                print >>log, header(datname)
                statqueue = (last_emptying_filter in filters) and -4 # exponential
                w, w3, w4 = run_main(plan, load=load, filters=filters, statqueue=statqueue)
                print >>log, 'total:', w
                print >>log, '3:', w3
                print >>log, '4:', w4
    log.close()
```